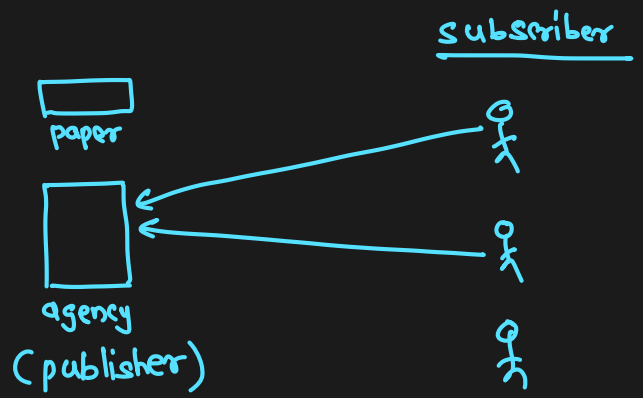
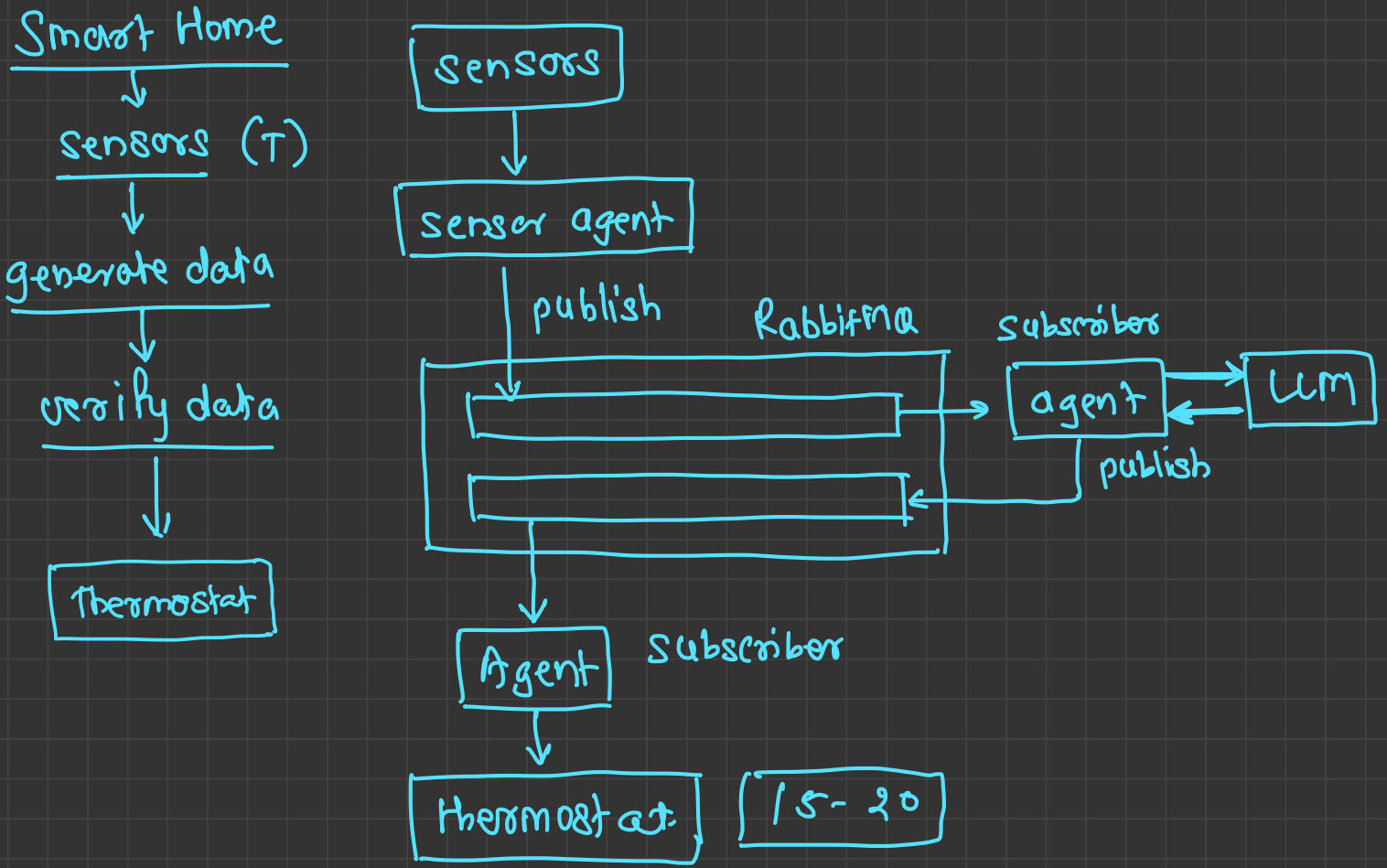




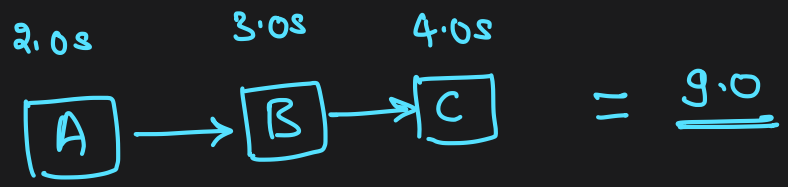
Event Driven Agents

publisher - subscriber pattern

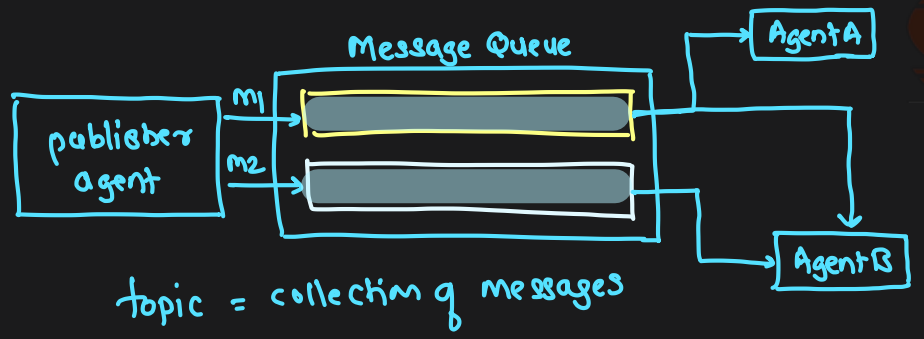

Smart Home
sensors (T)
generate data
verify data
Thermostat
sensors
sensor agent
publish
Rabbitmq
subscriber
agent
LLM
publish
Agent
subscriber
Thermostat
15-20

Sequential Pipeline Trap

- Each agent holds the execution stack open while waiting
- Agent C cannot begin work until B returns, even if C's task is independent
- The fastest agent remains bottlenecked behind the slowest
- Failure in any agent cascades up the entire chain
- Adding a new agent requires modifying the caller
- No parallelism possible within a single request
- Throughput dictated by the slowest agent
- Debugging requires tracing a single monolithic stack



Event Bus Architecture



- Agents publish typed events to named topics
- Agents subscribe to topics they care about
- No agent holds the execution stack open waiting for another
- When an agent completes work, it **publishes an event** rather than returning a value to a caller
- Downstream agents are **woken by their topic subscriptions**, not by explicit handoffs
- The broker handles routing, durability, and delivery guarantees
- Multiple agents can subscribe to the **same topic** and all receive the event simultaneously
- Publishers do not know who consumes their events
- Subscribers do not know who produces the events
- The bus becomes the single integration point
- New subscribers can be added without touching publishe

Core Characteristics

Asynchronous

- Agents operate independently, responding to triggers as they arrive
- The publisher does **not wait** for a response
- An agent can be offline; the event remains in the broker until it returns
- A single agent going down does **not halt the ecosystem**
- Publisher and subscriber lifecycles are independent
- Events can be processed at different times
- No shared execution thread between agents

Sequential	Event-Driven
Caller blocks until response	Publisher fires and continues
Failure cascades up the chain	Failure isolated to one subscriber
Tight temporal coupling	Loose temporal coupling
Both must be alive simultaneously	Either can be offline
Single point of failure	Redundant consumers possible
Synchronous error propagation	Errors handled per-subscriber
Limited concurrency	Natural concurrency

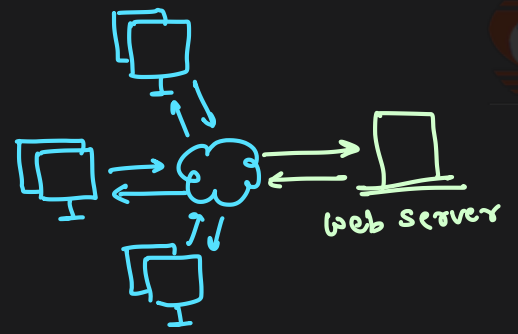
Decoupled

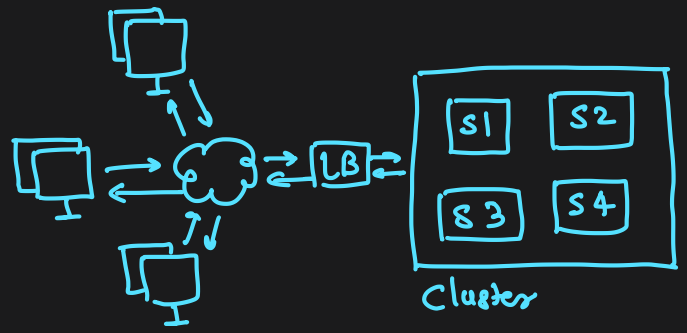


Dimension	Sequential	Event-Driven
Temporal	Must be alive simultaneously	Publisher/subscriber need not overlap
Spatial	Must know each other's address	Only need to know the topic name
Referential	A must know B's interface	A only knows event schema
Evolutionary	Changing B breaks A	Adding subscribers does not affect publisher
Protocol	Shared RPC contract	Shared event contract
Team	Cross-team coordination needed	Teams own their agents independently
Deployment	Coordinated releases	Independent deployment
Testing	Integration tests required	Contract tests sufficient

Fault Tolerance

- **Durability:** Events persist in the broker until acknowledged
- **Replay:** Event log is the source of truth; state can be reconstructed
- **Store-and-forward:** Agents with intermittent connectivity can catch up
- **Isolation:** One agent's failure does not cascade
- **Retry:** Failed processing can be retried without affecting other subscribers
- **Dead Letter Queue:** Failed events captured for later analysis
- **Idempotency:** Events can be processed multiple times safely
- **Checkpointing:** Progress tracked per consumer (subscriber)
- **Circuit Breakers:** Prevent cascading failures
- **Bulkheads:** Isolate failures to specific agent pools





Horizontal scaling

Scalability Through Parallelism



- Each agent type scales independently based on its own event load
- Horizontal scaling: add more instances of a slow agent
- System operates only as slowly as its single longest task
- Backpressure handled by broker (queue depth, consumer lag)
- Consumer groups enable load balancing across instances
- Partitioning enables ordered processing where needed
- Auto-scaling based on lag metrics
- No single point of contention

Implementation: Message Broker

Broker	Best For	Key Features
Apache Kafka	Enterprise scale, high throughput	Durability, replay, partitioning, exactly-once
AWS EventBridge	AWS-native, serverless	Routing rules, schema registry, SaaS integration
Google Pub/Sub	GCP-native, global scale	At-least-once, ordering, dead letter
Solace Agent Mesh	Multi-agent AI specifically	Built for agent transport, event mesh
asyncio queues	Python-native, lightweight	Minimal viable event bus, per-agent workers
Redis Streams	Low-latency, moderate scale	Consumer groups, persistence, simple ops
RabbitMQ	Complex routing, AMQP	Exchanges, queues, flexible routing
NATS	Cloud-native, lightweight	Fast, simple, JetStream for persistence
Azure Service Bus	Azure-native, enterprise	Sessions, transactions, dead letter
Pulsar	Multi-tenant, geo-replication	Tiered storage, functions, multi-protocol

Implementation: Dynamic Routing



- **Rule-Based Routing:** Event bridge style
- **Semantic Routing:** LLM-based routing
- **Content-based:** Route by payload fields
- **Header-based:** Route by metadata
- **Topic-based:** Route by event type
- **Priority-based:** Route urgent events first
- **Round-robin:** Distribute load evenly
- **Sticky:** Same correlation ID to same agent
- **Broadcast:** All subscribers receive
- **Fan-out/fan-in:** Parallel then aggregate

Common Pitfalls to Avoid

Pitfall	Symptom	Fix
Event explosion	Thousands of trivial events	Batch, aggregate, or use coarser event types
Schema drift	Subscribers break on new fields	Versioned schemas, backward compatibility
Lost correlation	Cannot trace flow across agents	Mandatory correlation_id in envelope
Blocking subscribers	One slow agent backs up the bus	Async processing, separate consumer groups
Infinite loops	Agent A triggers B triggers A	Idempotency keys, cycle detection, TTL
No dead letter queue	Failed events disappear silently	DLQ + alerting + replay tooling
Over-engineering	Kafka for 10 events/day	Match broker to scale; start simple
Poison messages	One bad event crashes consumer	Validation, DLQ, circuit breaker
Ordering assumptions	Events processed out of order	Partitioning by key, sequence numbers
Duplicate processing	Same event processed twice	Idempotency keys, deduplication
Unbounded queues	Memory exhaustion	TTL, max queue depth, backpressure
Missing observability	Cannot debug production	Traces, metrics, correlation IDs

When to Use

- Multiple agents can work in parallel on the same trigger
- Agents have different processing tempos
- Fault tolerance and replay are required
- System must scale horizontally
- New agents will be added over time without modifying existing ones
- Human-in-the-loop governance is needed
- Audit and compliance trails are required
- Agents are owned by different teams
- System must handle intermittent connectivity
- Workload is bursty or unpredictable

When to avoid

- Strict sequential ordering is mandatory (e.g., financial transaction steps)
- Ultra-low latency is required (<10ms) and broker overhead is prohibitive
- Team lacks operational maturity for distributed systems
- Simple 2–3 agent pipeline with no growth expected
- Strong consistency across agents is required (eventual consistency is unacceptable)
- Debugging simplicity is paramount
- Transactional guarantees span multiple agents
- Total system state fits in a single process